VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY

Chennai, India · Office of International Relations · Value Added Course

Explainable Artificial Intelligence

VTR UGE 21

DAY 1 · MODULE 1

Foundations of Interpretability

Why models need explanations, the map of explanation methods, models that explain themselves and a model that is accurate for the wrong reason.

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University
Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)
IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 · utkukose.com · github.com/utkukose
utkukose@sdu.edu.tr · utku.kose@und.edu · ukose@up.edu.mx · utkukose@gmail.com

Overview

Machine learning models now support decisions in medicine, finance and engineering. A model can be accurate and still be unacceptable if nobody can see why it decides as it does. This first day introduces explainable artificial intelligence (XAI): Why explanations are needed, how the field defines interpretability, how explanation methods are grouped and which models are readable by design [1, 2]. It closes with a model that scores very well and is right for the wrong reason, the kind of failure that explanations exist to reveal [3]. The running example of the course is the diagnosis of breast tumours from measurements of cell nuclei [4].

Estimated study time: 6 to 8 hours.

Learning outcomes

By the end of the day, students are expected to explain why accurate models still need explanations, to define interpretability and explainability, to place an explanation method on the map of global and local, built-in and post-hoc methods, to read a logistic regression and a decision tree, to discuss the trade-off between accuracy and readability, to name the properties of a good explanation and to recognise a model that learned a shortcut. In Python, students are expected to use values, variables, lists, decisions, functions and dictionaries.

Python in this day

The Colab notebook opens with Python step 1: Values, variables, lists and decisions, right after its setup section. It covers numbers and text, lists, `if` statements, `for` loops, functions and dictionaries, applied to a threshold rule for five patients [5]. Students who have never programmed should first open the start-here notebook of the course.

How these notes point to the Python code. Every method of this lecture is also written as Python code in the Colab notebook of the day, the file `NB01_foundations_of_interpretability.ipynb`. A Colab notebook is a document of text and Python code that runs in the web browser, with nothing to install. It opens with the button Open in Colab on the day overview page of the course, https://github.com/utkukose/xai-veltech-NB-lecture/tree/main/days/day-01. The code of the notebook is divided into numbered sections. In these notes, each box titled *Python code in the Colab notebook, Section N* names the section whose code carries out what the text above the box explains, and says what to run and what to look at there. The animations that the notes mention are interactive figures on the lecture page of the day, https://utkukose.github.io/xai-veltech-NB-lecture/days/day-01/lecture.html. The interactive lab, with the practice parts and the self-assessment of the day, is at https://utkukose.github.io/xai-veltech-NB-lecture/days/day-01/lab.html. Each box titled *Interactive lab, Part X* opens one part of the lab. The part ends with a link back to the lecture page, and the notebook is always opened from the boxes of the lecture.

Python step 1: Values, variables, lists and decisions

A notebook is a sequence of cells. Text cells explain, and code cells hold Python instructions that run when the cell is executed with Shift+Enter, with the output shown directly below the cell [21]. This step writes the simplest interpretable model there is, a single threshold rule, and evaluates it on five real tumours from the dataset of the day [4, 5].

Values and variables

```
radius = 17.99            # mean radius of the cell nuclei of one tumour
concave_points = 0.1471   # mean concave points of the same tumour
diagnosis = "malignant"   # label recorded by the pathologist
print(type(radius), type(diagnosis))
print("radius:", radius, "concave points:", concave_points)
```

Output

```
<class 'float'> <class 'str'>
radius: 17.99 concave points: 0.1471
```

A variable is a name bound to a value with the operator `=`. Every value has a type: Numbers with a decimal point are floats, whole numbers are integers, text in quotation marks is a string, and `True` and `False` are booleans. The function `print` displays values, and text after `#` is a comment that Python ignores.

A decision is an if statement

```
threshold = 0.05
if concave_points > threshold:
    prediction = "malignant"
else:
    prediction = "benign"
fired = concave_points > threshold
print(f"Rule: concave points > {threshold} means malignant")
print(f"Value {concave_points:.4f}, rule fired: {fired}, prediction: {prediction}")
```

Output

```
Rule: concave points > 0.05 means malignant
Value 0.1471, rule fired: True, prediction: malignant
```

An `if` statement runs the indented block below it only when its condition is true, and `else` covers the remaining case. Indentation is part of the syntax. The comparison `concave_points > threshold` is itself a boolean value. An f-string, written with the letter `f` before the quotation mark, inserts the value of each expression in braces, and `:.4f` prints a float with four decimals. This rule is a complete classifier, and its explanation is its own source code.

Lists and loops

```
values = [0.1471, 0.0702, 0.0478, 0.0208, 0.0203]
labels = ["malignant", "malignant", "benign", "benign", "malignant"]
correct = 0
for value, truth in zip(values, labels):
    guess = "malignant" if value > threshold else "benign"
    correct = correct + (guess == truth)
    print(f"{value:.4f} -> {guess:9s} truth: {truth}")
print(f"accuracy: {correct} of {len(values)} = {correct / len(values):.2f}")
```

Output

```
0.1471 -> malignant truth: malignant
0.0702 -> malignant truth: malignant
0.0478 -> benign    truth: benign
0.0208 -> benign    truth: benign
0.0203 -> benign    truth: malignant
accuracy: 4 of 5 = 0.80
```

A list holds values in order between square brackets, and `len` returns its length. The `for` loop runs its block once per element, and `zip` walks two lists in step: `for value, truth in ...` gives the two values of each step the names `value` and `truth`. The one-line form `a if condition else b` chooses between two values. Adding a boolean to a number counts it as 1 or 0. In the f-string, `:9s` pads the text to nine characters, so that the columns line up. The last tumour is malignant with few concave points, so the rule misses it: An interpretable model can still be wrong, and its error is visible.

Functions

```
def threshold_rule(value, limit=0.05):
    """A one-feature glass-box classifier."""
    return "malignant" if value > limit else "benign"

def accuracy_of(limit):
    hits = [threshold_rule(v, limit) == t for v, t in zip(values, labels)]
    return sum(hits) / len(hits)

for limit in [0.01, 0.02, 0.05, 0.08]:
    print(f"limit {limit:.2f}: accuracy {accuracy_of(limit):.2f}")
```

Output

```
limit 0.01: accuracy 0.60
limit 0.02: accuracy 0.60
limit 0.05: accuracy 0.80
limit 0.08: accuracy 0.60
```

The keyword `def` defines a function with parameters, an optional default value such as `limit=0.05` and a `return` statement. The text in triple quotation marks is a docstring that documents the function. The expression in square brackets inside `accuracy_of` is a list comprehension, a compact loop that builds a list. Searching over thresholds is a first, very small form of model training.

Dictionaries and a rule list

```
rules = {"mean concave points": 0.05, "worst radius": 16.8}
patient = {"mean concave points": 0.0203, "worst radius": 15.93}
reasons = []
for feature, limit in rules.items():
    if patient[feature] > limit:
        reasons.append(f"{feature} = {patient[feature]} > {limit}")
decision = "malignant" if reasons else "benign"
print("decision:", decision)
print("because:", "; ".join(reasons) if reasons else "no rule fired")
```

Output

```
decision: benign
because: no rule fired
```

A dictionary maps keys to values between curly braces, and `patient[feature]` looks a value up by its key. The method `items` returns key and value pairs for a loop, and `append` adds an element to a list. In a condition, a list counts as true when it holds at least one element, so `"malignant" if reasons else "benign"` decides by whether a rule fired, and `"; ".join(reasons)` joins the reasons into one text. The program is a rule list whose explanation, the list `reasons`, is produced together with the decision. Here no rule fires, and the rule list repeats the error of the single threshold for the same tumour.

Quick check. What does `print(7 > 5 and 2 > 3)` display?

Why explain a model?

A machine learning model is a rule that turns inputs into a prediction. Its details are not written by a programmer but found from examples, a process called training. The running example of the course is medical: A sample of cells is taken from a lump in the breast, a tumour, and measured under a microscope. A tumour is either malignant, which means cancerous, or benign, which means harmless. The share of right answers of a model on cases that were kept apart from training, the test data, is called its accuracy.

> **Machine learning and deep learning**
>
> Artificial intelligence is the wider field of computer systems that carry out tasks which call for human judgement. Machine learning is the part of artificial intelligence in which a model finds its rule from examples. Deep learning is machine learning with deep neural networks: models built from many layers of simple computing units, which Day 3 opens up. It is the form of machine learning behind most of today's systems for images, speech and language, including the language models of chat assistants.
>
> A deep network is therefore a machine learning model as well, trained from examples like a tree or a regression, and everything that this course says about machine learning models holds for it. The model-agnostic methods of Day 2 explain it in the same way as any other model, and Day 3 adds methods that read its inside.

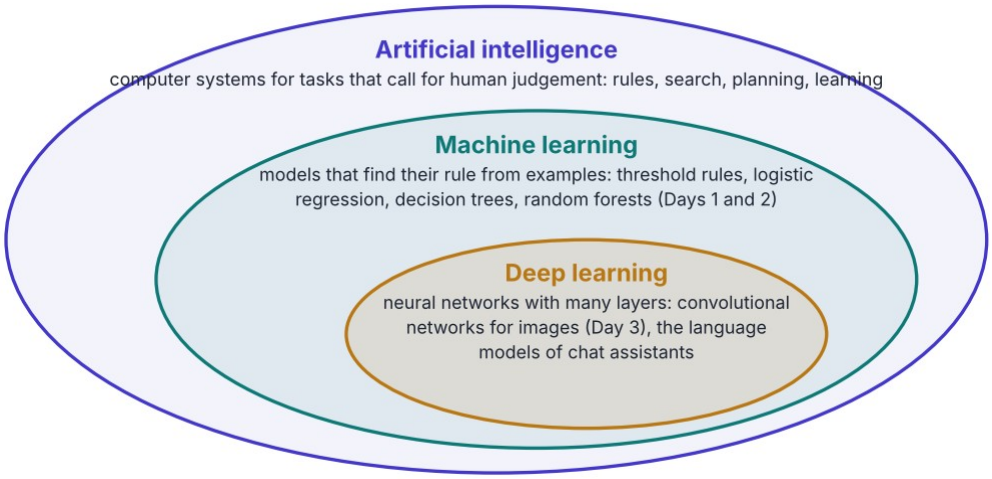


Figure. Deep learning is a part of machine learning, and machine learning a part of artificial intelligence. Each oval names the models of the course that belong to it.

Check your understanding (1). A hospital uses a deep convolutional network to read X-ray images. Which statement is right?

A. It is deep learning and therefore not machine learning

B. It is a machine learning model of the deep learning kind, trained from examples

C. It is a set of rules written by programmers

D. The methods of this course cannot explain it

Suppose a model reads measurements of a breast tumour and predicts whether it is malignant. Its accuracy on test data is 97 percent. A doctor who uses it still has questions: Which measurements led to this prediction? Would a slightly different measurement change it? Did the model learn something medically meaningful, or a quirk of the data? Accuracy answers none of these questions.

The diagnosis case, the project of the course

Answering these questions for one diagnostic task is the project of the course. The task is the diagnosis of breast tumours from measurements of cell nuclei. An imaged version of it adds lesion images in which a hidden marker leaks the label, and because the marker is planted on purpose, every explanation of these images can be graded against the truth.

Each day adds one part of an explanation audit of these models. Day 1 builds readable models of the diagnosis and meets a model that is accurate for the wrong reason. Day 2 explains a random forest that cannot be read, for the whole model and for one patient. Day 3 trains a deep network on the lesion images and asks where it looks. Day 4 tests whether the probabilities, the explanations and the decisions can be trusted, with a credit model as a second case. Day 5 turns the results into artefacts for the clinic: a record, a dashboard and a model card. The final project asks for the same audit of a model that the student chooses.



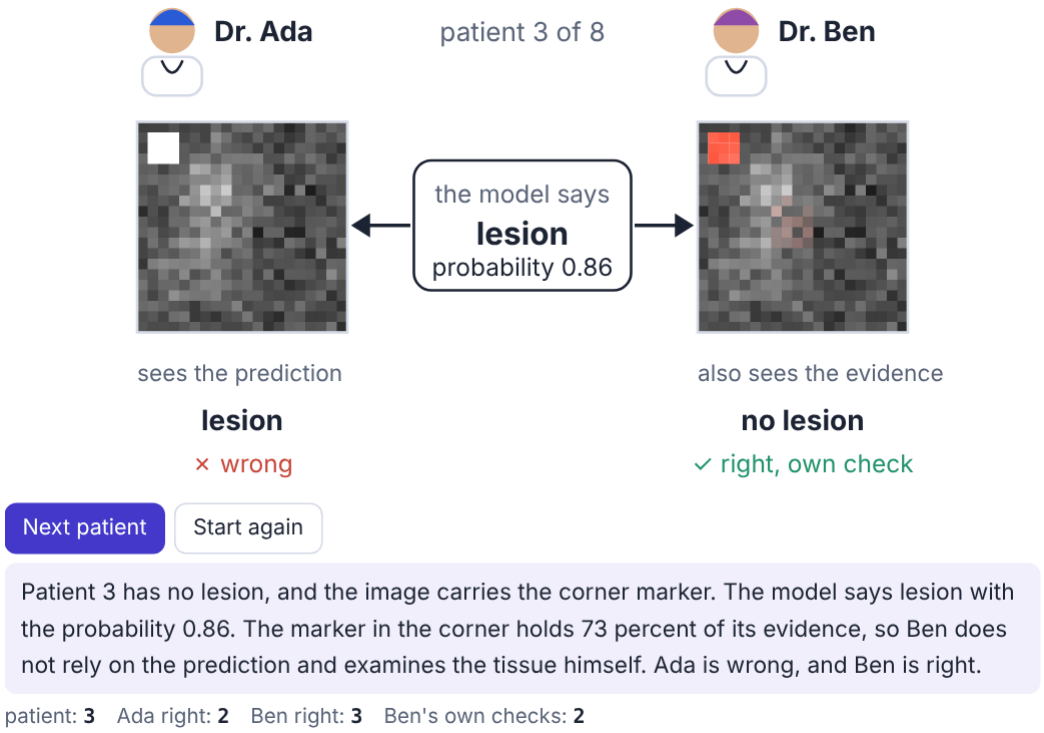
Figure. The five stages of the project through the week. Today readable models of the diagnosis are built, and a model is caught being right for the wrong reason.

Explanations serve several groups of people with different needs [1]. Developers use them to find errors and hidden shortcuts. Users such as doctors or engineers need them to decide when to trust a prediction. People affected by a decision, such as a rejected loan applicant, may have a right to understand it. Regulations such as the Artificial Intelligence Act of the European Union ask for transparency in high-risk applications [6]. Explanations also support scientific discovery, when a model points to a relation that experts had not noticed [7].

A person or a group with an interest in the decisions of a model is called a stakeholder. Stakeholders ask for explanations out of four motives, and each motive is met by a different product, called an artefact in this course: a chart, a sentence or a stored record.

Motive	Who asks	Question	Artefact that answers it
Trust	A user, such as a doctor	Should this prediction be acted upon?	The reasons for this prediction, such as a chart of the inputs that pushed it
Debugging, the search for the cause of an error	A developer	Why is this case wrong?	A view of what the model relies on, such as a ranking of its inputs
Recourse, a way to obtain a different outcome	A person affected by a decision	What must change?	A sentence that names a small change the person can make
Compliance, meeting the rules of a law or a standard	An auditor or a regulator	Can the decision be documented and repeated?	A stored record of the model, the inputs and the output

An artefact that serves one motive can be useless for another: A ranking of the inputs helps a developer and tells a rejected applicant nothing that she can act on.



Interactive scene on the lecture page: Two doctors, one model. Two doctors use the same model on eight patients. The model reads lesion images like those of the course, with 20 by 20 pixels, and was trained in this page on 600 images in which a marker in the top left corner came with 97 of 100 lesions, the shortcut of the section Accurate for the wrong reason below. Dr. Ada sees the prediction only. Dr. Ben also sees, in red, the pixels that pushed the model towards a lesion, and he examines the tissue himself when this evidence lies on the marker. The readouts count how often each doctor is right.

> Check your understanding (2). A model has 97 percent test accuracy. Why can it still need an explanation?
>
> A. It cannot
>
> B. Accuracy does not show why it decides, whether it uses meaningful features or when it fails
>
> C. Because accuracy is always overestimated
>
> D. Because the data are too small

The words of this section have fixed symbols in the course. One case, here one tumour, is described by d numbers, called features or measurements: $x = (x_1, \ldots, x_d)$, where x_j is feature number j. The correct answer for a case is its label y, and the possible labels are the classes, here malignant and benign. A model is a function f, and its prediction for the input x is written $\hat{y} = f(x)$, with a hat for a predicted value. A model that predicts a class is called a classifier. The cases with a known label are split into the training data, from which the model is built, and the test data. As a formula, accuracy is

$$\text{accuracy} = \frac{\text{number of correct predictions}}{\text{number of cases}}$$

A model that is right for 139 of 143 test tumours has an accuracy of 139 / 143 = 0.972, or 97 percent.

The data of the day are the Breast Cancer Wisconsin (Diagnostic) data: 569 tumours, 212 malignant and 357 benign [4]. For every tumour, ten properties of the cell nuclei were measured on a microscope image: radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension [8]. Concave points are the dents in the outline of a nucleus. Each property enters three

times: as its mean over the nuclei of the image, as its standard error and as its *worst* value. The standard error grows with the spread among the nuclei, and the worst value is the mean of the three largest values. This gives $d = 30$ features with names such as `mean radius`, `radius error` and `worst concave points`. The label is a number, $y = 0$ for malignant and $y = 1$ for benign. Of the 569 tumours, 426, that is 426 / 569 = 0.749 or 75 percent, serve as training data and the other 569 − 426 = 143 as test data. Both parts keep the share of malignant tumours of the whole data, 212 / 569 = 0.373, 37 percent: 159 of the 426 training tumours and 53 of the 143 test tumours are malignant.

Two kinds of plot show how the classes differ in a feature. A histogram counts how many tumours fall into each range of values, and a scatter plot shows every tumour as a point with one feature on each axis. The benign tumours have a mean `worst radius` of 13.4 and the malignant ones of 21.1, but the two groups overlap between 12.8 and 19.8.

> **Python code in the Colab notebook, Section 1.**
>
> Open Section 1 of the Colab notebook and run it. It loads the 569 tumours, splits them into training and test data with `train_test_split`, prints the number of tumours of each class and draws two histograms and a scatter plot.

Interpretability and explainability

Doshi-Velez and Kim define interpretability as the ability to explain or present a model in terms that a human can understand [2]. Two words are used for this idea. A model is called interpretable when a person can follow how it works directly, as with a short list of rules. An explanation, in contrast, is something produced about a model, for example a chart of the most influential inputs. Lipton warns that both words are used loosely, and that a claim of interpretability should always say for whom and for what purpose [9].

The course uses the two words as follows. Interpretability is a property of the model itself: A person can inspect the model and predict what it will do. Such a model is also called a glass box. Explainability is the ability to produce explanations with a method that is applied after training, also for a model that cannot be read, a black box. A black box need not be secret: The random forest of this day can be printed in full, and its 10344 decision nodes are more than any reader can hold in mind. An explanation of a black box is a second, simpler description of the model and in general only an approximation.

A map of explanation methods

Explanation methods are usually placed on three axes [1, 10]. The first asks whether the explanation covers the whole model or one prediction. The second asks whether the model is readable by design or the explanation is computed after training. The third asks whether a method works for any model or only for one family, such as neural networks.

A neural network is a model built from many simple computing units arranged in layers, and a network with many layers is called deep. A saliency map is an image that marks how strongly each pixel of an input image influenced a prediction [11]. Gradient-weighted class activation mapping, abbreviated Grad-CAM, marks the regions of an image that a network used [12]. Shapley additive explanations, abbreviated SHAP, split one prediction into shares of the inputs [13]. Coefficient is another word for a weight of a model.

Axis	Options	Example
Scope	global: the whole model; local: one prediction	the coefficients of a model; the reasons for one decision
Origin	built in by design; computed after training (post-hoc)	a small decision tree; a saliency map
Reach	model-specific; model-agnostic	Grad-CAM for networks; SHAP for any model

The Latin words post hoc mean after the event, here after training. Agnostic means not knowing: A model-agnostic method does not look inside the model and only asks it for predictions, so it works for any model. A model-specific method reads the inner parts of one family of models.

The cards of Part A of the interactive lab use four further forms of explanation, which Day 2 builds. An attribution gives every input a number that states how much it contributed to a prediction, written φ_j for feature j, with the Greek letter phi. A permutation importance shuffles the values of one input and records how much the accuracy of the model falls. A partial dependence plot draws the average prediction against the values of one input. A counterfactual names a small change of the inputs that would have changed the decision [14].

Day 2 covers model-agnostic methods, Day 3 methods for deep networks, and Days 4 and 5 the reliability and use of explanations.

> **Interactive lab, Part A.**
>
> Eight explanations from practice, each to be placed on the map: one prediction or the whole model, readable by design or computed after training. Part A opens at https://utkukose.github.io/xai-veltech-NB-lecture/days/day-01/lab.html#pa.

> Check your understanding (3). A bank tells a rejected applicant which three inputs lowered her score most, computed after training. Where does this explanation sit on the map?
>
> A. Global and built in
>
> B. Local and post-hoc
>
> C. Global and post-hoc
>
> D. Local and built in

Models that explain themselves

Some models are readable by design. The smallest one, and the first model of the diagnosis case, is a threshold rule. It compares one feature x_j with a fixed number, the threshold t:

$$\hat{y} = \begin{cases} \text{benign} & \text{if } x_j \le t \\ \text{malignant} & \text{if } x_j > t \end{cases}$$

For the tumours the feature is `worst concave points`. Training this model means choosing the threshold: 32 values from 0.06 to 0.215 are tried, and the one with the highest accuracy on the training data is kept.

$$t^* = \arg\max_t \ \text{accuracy}_{\text{train}}(t)$$

The symbol arg max returns the threshold at which the accuracy is largest, while max would return that accuracy itself. The result is 0.14, with a training accuracy of 0.918, and the rule can be said in one sentence: A tumour is benign when its worst concave points are at most 0.14. The errors of a classifier are counted in a confusion matrix, a table with one row for every true class and one column for every predicted class. For the rule on the test data it reads:

True class	Predicted malignant	Predicted benign
Malignant, 53 test tumours	45	8
Benign, 90 test tumours	7	83

Worked example: accuracy from a confusion matrix

The correct predictions stand on the diagonal of the table: 45 malignant tumours that the rule calls malignant and 83 benign tumours that it calls benign. The accuracy is (45 + 83) / 143 = 128 / 143 = 0.895.

The two kinds of error are not equally serious. The rule calls 8 of the 53 malignant tumours benign, 8 / 53 = 0.151 or 15 percent of them, and 7 of the 90 benign tumours malignant, 7 / 90 = 0.078 or 8 percent. A single accuracy hides this difference.

In everyday life: two kinds of error

A smoke alarm can fail in two ways. It can sound when someone burns the toast, or it can stay silent during a fire. Both are errors, but nobody weighs them equally, and a good alarm accepts some false alarms in order to miss no fire. A diagnosis has the same two kinds of error, a missed cancer and a false alarm, and the confusion matrix keeps them apart.

Python code in the Colab notebook, Section 2.

Open Section 2 of the Colab notebook and run it. It holds the threshold rule as the function `rule` and finds the best threshold with `np.argmax(train_acc)`. Its first cell draws the accuracy for every threshold and the confusion matrix and prints the test accuracy of the best rule.

A logistic regression adds up the inputs, each multiplied by a weight, and turns the sum into a probability. The features have very different sizes: `worst area` reaches values in the thousands, and `worst concave points` stays below 0.3. Every feature is therefore first put on a common scale, a step called standardisation:

$$\tilde{x}_j = \frac{x_j - m_j}{s_j}$$

$$m_j = \frac{1}{n}\sum_{i=1}^{n} x_j^{(i)}$$

$$s_j = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(x_j^{(i)} - m_j\right)^2}$$

Here $x_j^{(i)}$ is the value of feature j for training tumour i, $n = 426$ is the number of training tumours, and the sign Σ means: Add the term for $i = 1, 2, \ldots, n$. The mean of the feature is m_j, and s_j is its standard deviation,

the usual measure of how far the values lie from their mean. The scaled value $\tilde{x_j}$, written with a tilde, counts how many standard deviations a measurement lies above or below the mean. For `worst radius`, $m_j = 16.20$ and $s_j = 4.80$, so a worst radius of 24.30 becomes (24.30 − 16.20) / 4.80 = 8.10 / 4.80 = 1.69.

The model then forms a weighted sum of the scaled values, the score z, and turns it into a probability p with the logistic function, written with the Greek letter sigma:

$$z = b + \sum_{j=1}^{d} w_j \tilde{x}_j$$

$$p = P(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The weights $w_1, \ldots, w_d$ and the bias b are the numbers that training finds, 31 values in all. The vertical bar reads *given*: $P(y = 1 \mid x)$ is the probability that the tumour is benign, given its measurements. The number $e = 2.718$ is the base of the natural logarithm, written ln. The logistic function maps every score to a number between 0 and 1: A score of 0 gives $1 / (1 + e^0) = 1 / 2 = 0.5$. A score of 2 gives $1 / (1 + e^{-2}) = 1 / (1 + 0.135) = 1 / 1.135 = 0.88$, and a score of −2 gives $1 / (1 + e^2) = 1 / (1 + 7.389) = 1 / 8.389 = 0.12$. The model predicts benign when p is above 0.5, that is, when z is positive.

The odds of an event are its probability divided by the probability of the opposite, $p / (1 - p)$: A probability of 0.8 means odds of 0.8 / 0.2 = 4, or 4 to 1. Solving the logistic function for z shows that the score is the natural logarithm of the odds, the log-odds:

$$\ln \frac{p}{1 - p} = z = b + \sum_{j=1}^{d} w_j \tilde{x}_j$$

After the inputs are put on a common scale, each weight shows how strongly a measurement pushes a prediction towards one class. When feature j grows by one standard deviation and the others stay fixed, the log-odds of benign change by w_j, and the odds are multiplied by e^{w_j}. A negative weight pushes towards malignant and a positive weight towards benign. The model is its own explanation.

> **In everyday life: a score of points**
>
> Every newborn baby is checked one minute and five minutes after birth with the Apgar score. Five signs, heart rate, breathing, muscle tone, reflexes and skin colour, receive 0, 1 or 2 points each, and the sum from 0 to 10 guides the next steps. Every point can be traced back to a sign. A logistic regression is a score of the same kind, with weights found from data instead of fixed points, and it can be checked in the same way.

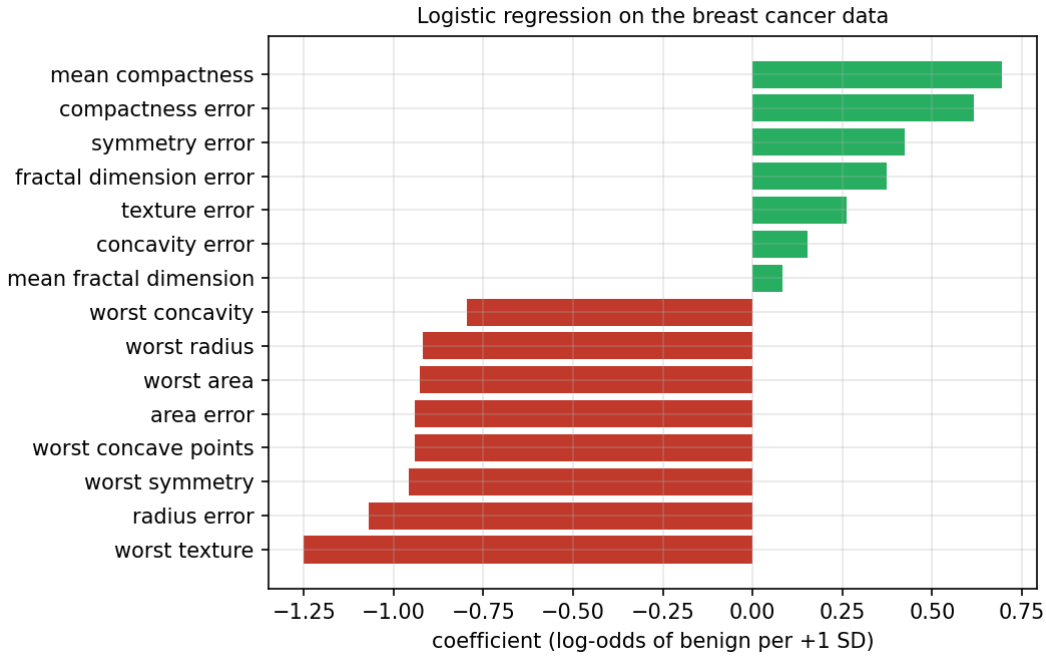


Figure. The eight most negative and the seven most positive of the 30 weights of a logistic regression on the tumour data, with the inputs on a common scale. The axis shows the change of the log-odds of benign when a feature grows by one standard deviation (SD).

Red bars are negative weights and green bars positive ones. The longest bar belongs to `worst texture` with −1.25: One standard deviation more of this feature multiplies the odds of benign by $e^{-1.25} = 0.29$. It is followed by `radius error` with −1.07 and by five weights between −0.96 and −0.92, among them `worst radius`, `worst area` and `worst concave points`. Of the 30 weights, 22 are negative, and the bias is $b = 0.28$. The model classifies 141 of the 143 test tumours correctly, an accuracy of 141 / 143 = 0.986.

> **Remark: a weight holds the other features fixed**
>
> A weight states what happens when one feature changes and all others stay as they are. Features that rise and fall together, such as `mean radius`, `mean perimeter` and `mean area`, are called correlated, and they share one effect among their weights in a way that can look odd. Malignant tumours have the higher `mean compactness` on average, 0.145 against 0.080, and yet its weight is positive, +0.69. The weights describe the model exactly. They do not describe each feature on its own.

The same sum explains a single prediction. The product of a weight and the scaled value of one patient is the contribution of that feature to the score, an attribution in the sense of the map of methods:

$$\phi_j = w_j \, \tilde{x}_j, \qquad z = b + \sum_{j=1}^{d} \phi_j$$

The weights are a global explanation, the same for all patients, and the contributions are a local explanation of one prediction. Together with the bias, the contributions add up to the score exactly, so this explanation leaves nothing of the model out.

Worked example: one prediction of a logistic regression

A model with three features has the bias $b = 0.3$ and the weights −0.9, −1.2 and 0.7. A patient has the scaled values 2.0, 0.5 and 1.0. The contributions are $-0.9 \times 2.0 = -1.8$, $-1.2 \times 0.5 = -0.6$ and $0.7 \times 1.0 = 0.7$, and the score is $z = 0.3 - 1.8 - 0.6 + 0.7 = -1.4$.

The probability of benign is $1 / (1 + e^{1.4}) = 1 / (1 + 4.06) = 1 / 5.06 = 0.20$, so the model predicts malignant, and the first feature is the main reason. The odds of benign are $0.20 / (1 - 0.20) = 0.20 / 0.80 = 0.25$, one to four, the same as $e^{z} = e^{-1.4} = 0.25$. In this and the following worked examples, intermediate values are rounded, and each step continues with the rounded value.

Interactive lab, Part B.

The formulas of this section solved step by step with numbers: the contributions, the score, the probability, the odds and the main reason of the same prediction, with sliders for the scaled values of the patient and the bias. Part B opens at https://utkukose.github.io/xai-veltech-NB-lecture/days/day-01/lab.html#pb.

Animation on the lecture page: A spam filter is a logistic regression on the words of an e-mail. Choose an e-mail, click a highlighted word to remove it or add a word below, and watch every word add its contribution to the score.

The filter has a bias of −1.5 and six weights: 1.8 for *winner*, 1.2 for *free*, 1.0 for *urgent*, 0.9 for *click*, −1.0 for *report* and −1.3 for *meeting*. Every occurrence of a word adds its weight, so the contribution of a word is its weight times its count, the $\varphi_j = w_j x_j$ of the tumour model with counts in place of scaled measurements. The first e-mail contains four of the words: $z = -1.5 + 1.8 + 1.2 + 1.0 + 0.9 = 3.4$ and $p = 1 / (1 + e^{-3.4}) = 0.968$, so it lands in the spam folder, mainly because of *winner*. The last e-mail lies close to the boundary: $z = -1.5 + 1.0 + 0.9 - 1.0 = -0.6$ and $p = 0.354$. Without the word *report* the score rises to 0.4 and the probability to 0.599, and the e-mail becomes spam: One word decides, and the contributions show which one.

For test patient 5, the six largest of the 30 contributions lie between −1.55 for `worst area` and −1.15 for `worst perimeter`, and all 30 add up to −14.13. The score is $z = 0.28 - 14.13 = -13.85$. With $e^{13.85} = 1.04 \times 10^{6}$, the probability of benign is $1 / (1 + 1.04 \times 10^{6})$, about one in a million, 0.00 when rounded to two decimals. The diagnosis of this patient is malignant.

How the weights are found. Training searches for the weights with which the predicted probabilities fit the labels of the training tumours. The misfit for tumour i is the log loss $\ell^{(i)}$, written with a script letter l, and training makes the function J as small as possible:

$$\ell^{(i)} = -\left[y^{(i)} \ln p^{(i)} + \left(1 - y^{(i)}\right) \ln \left(1 - p^{(i)}\right) \right]$$

$$J(w, b) = C \sum_{i=1}^{n} \ell^{(i)} + \frac{1}{2} \sum_{j=1}^{d} w_j^2$$

Here $y^{(i)}$ is the label of the tumour, 1 or 0, and $p^{(i)}$ the predicted probability of benign. For a benign tumour the loss is $-\ln p^{(i)}$: A probability of 0.9 costs $-\ln 0.9 = 0.11$, and a probability of 0.1 costs $-\ln 0.1 = 2.30$. The second sum of J is a penalty on large weights, called regularisation, which makes the model depend less on the particular training tumours. The constant C sets the balance between fit and penalty, and the model of the day keeps the standard value $C = 1$ of the scikit-learn library [15].

No formula gives the minimum directly. It is found step by step: All weights start at zero, and every step moves each weight a little in the direction that lowers the loss. For the mean of the losses, without the penalty, one step of this method, called gradient descent, is

$$w_j \leftarrow w_j - \alpha \frac{1}{n} \sum_{i=1}^{n} \left(p^{(i)} - y^{(i)} \right) \tilde{x}_j^{(i)}$$

The arrow means that the value on the left is replaced by the value on the right. The Greek letter alpha, α, is the step size, called the learning rate, and $p^{(i)} - y^{(i)}$ is the error of the current model for tumour i. The bias is updated in the same way, with 1 in the place of $\tilde{x}_j^{(i)}$. The interactive lab trains its linear score in exactly this way, with $\alpha = 0.4$ and 600 steps. Scikit-learn uses a faster method of the same kind and stops when further steps no longer lower J.

> Check your understanding (4). A logistic regression has the bias 0.5 and the weights −2 and 1. A patient has the scaled values 1.0 and 0.5. What is the predicted probability of the class 1?
>
> A. About 0.73
>
> B. About 0.50
>
> C. About 0.27
>
> D. About 0.12

> **Python code in the Colab notebook, Section 3.**
>
> Open Section 3 of the Colab notebook and run it. The model `logit` is a `StandardScaler` followed by a `LogisticRegression`, and the function `explain_patient` multiplies the scaled values of one patient by the weights `coef_[0]`. Move the slider to choose a test patient: The bar chart shows the six largest contributions and the predicted probability.

A decision tree asks a sequence of yes or no questions about the inputs, and every prediction follows one path from the top question to a leaf. Each question compares one feature with a threshold, $x_j \le t$. A case that answers yes goes to the left, a case that answers no to the right, and a value equal to the threshold counts as yes. Every question and every final answer is a node: The top question is the root, and a node without a further question is a leaf. The depth of a tree is the largest number of questions on one path. A leaf predicts the class that most of its training cases have.

How a tree is grown. A tree is built from the training data, one question at a time. A good question separates the classes: After it, each side should hold mostly one class. How mixed a node is, is measured by its Gini impurity G, where p is the share of benign tumours among the training tumours of the node:

$$G = 1 - p^2 - (1 - p)^2 = 2\,p\,(1 - p)$$

A node with one class only is called pure and has $G = 0$, and a node with half of each class has the largest value, 0.5. A question sends n_{left} tumours of a node to the left and n_{right} to the right, and its quality is the weighted mean of the two impurities:

$$G_{\text{split}} = \frac{n_{\text{left}}\, G_{\text{left}} + n_{\text{right}}\, G_{\text{right}}}{n_{\text{left}} + n_{\text{right}}}$$

The algorithm tries every feature and every threshold midway between two neighbouring values of the training data and keeps the question with the smallest G_{split}. It then repeats the search separately for the left and the right part. A node is not split further when it is pure or when the allowed depth is reached, here a depth of 3. The procedure is known as CART, for classification and regression trees.

In everyday life: twenty questions

In the game of twenty questions, one player thinks of an animal and the other may ask only questions that are answered with yes or no. A good first question is whether the animal lives in water, not whether it is a cat, because either answer rules out a large part of the possible animals. A tree chooses its questions in the same spirit: The best question leaves two groups that are each as close as possible to a single class.

Worked example: the first question of the tree

The root of the tree in the figure below holds the 426 training tumours, 159 malignant and 267 benign. The share of benign is $p = 267 / 426 = 0.627$, and the impurity is $G = 2 \times 0.627 \times 0.373 = 0.468$.

The question `worst radius <= 16.795` sends 284 tumours to the left, 25 malignant and 259 benign, and 142 to the right, 134 malignant and 8 benign. On the left the share of benign is 259 / 284 = 0.912, and the impurity is $2 \times 0.912 \times 0.088 = 0.161$. On the right the share is 8 / 142 = 0.056, and the impurity is $2 \times 0.056 \times 0.944 = 0.106$. The weighted mean is $(284 \times 0.161 + 142 \times 0.106) / 426$, which is $(45.72 + 15.05) / 426 = 60.77 / 426 = 0.143$. One question lowers the impurity from 0.468 to 0.143, about 0.14, and no other question reaches a lower value.

Interactive lab, Part C.

The two impurity formulas solved step by step with numbers for the first question of the tree: the impurities of the root, of both sides and of the split, with sliders for the numbers of malignant and benign tumours sent to the left. Part C opens at https://utkukose.github.io/xai-veltech-NB-lecture/days/day-01/lab.html#pc.

A tree of depth three can be printed on one page and checked by an expert. Deeper trees fit the data better but quickly become too large to read, as the animation shows.

Animation on the lecture page: A decision tree on simulated tumours with two measurements, each scaled to the range from 0 to 1. Increase the depth and watch the test accuracy, the number of leaves L and the number of values a reader must follow, $2L - 1$.

The animation grows a tree on 300 simulated tumours and tests it on 120 others, and every question appears as a straight border between the coloured regions. Its count of the values to read is the number of nodes. A tree with L leaves has $L - 1$ questions, and a tree of depth D has at most 2^D leaves, because every level can double their number:

$$\text{nodes} = 2L - 1, \qquad L \le 2^D$$

From depth 1 to depth 7 the tree of the animation grows from 2 to 29 leaves and from 3 to 57 nodes, while its test accuracy stays between 83 and 89 percent.

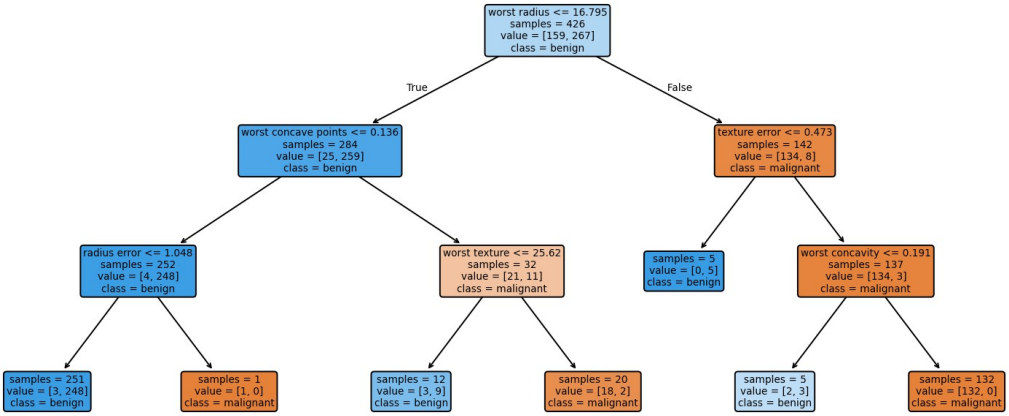


Figure. A decision tree of depth three for the tumour data. Every prediction follows at most three readable questions.

Each box of the figure is a node. Its first line is the question, and `samples` is the number of training tumours that reach the node. The line `value` lists how many of them are malignant and how many benign, and `class` names the majority, which a leaf predicts. Orange boxes hold mostly malignant and blue boxes mostly benign tumours. The tree has 6 questions and $L = 7$ leaves, so $2 \times 7 - 1 = 13$ nodes, and it classifies 135 of the 143 test tumours correctly, an accuracy of 135 / 143 = 0.944.

> **Worked example: the path of one patient**
>
> Test patient 10 has a worst radius of 16.67, worst concave points of 0.141 and a worst texture of 21.51. At the root, 16.67 ≤ 16.795 is true, so the path goes left. At the next node, 0.141 ≤ 0.136 is false, so the path goes right. At the third node, 21.51 ≤ 25.62 is true, so the path goes left and ends in a leaf.
>
> The leaf holds 12 training tumours, 3 malignant and 9 benign, and predicts benign. The explanation of the prediction is the path itself: worst radius at most 16.795, worst concave points above 0.136 and worst texture at most 25.62. The leaf also shows how firm the answer is: 9 of 12 training tumours, 9 / 12 = 0.75 or 75 percent, are benign.

> **Interactive lab, Part D.**
>
> Five patients to be followed by hand through a small tree, from the top question to a leaf. Part D opens at https://utkukose.github.io/xai-veltech-NB-lecture/days/day-01/lab.html#pd.

A path that can be read is not automatically a path that can be trusted: Test patient 94 ends after two questions in a leaf with only five training tumours, all benign, and the tree predicts benign for a malignant tumour. Depth has a price as well. From depth 1 to depth 6, the tree grows from 3 to 33 nodes. Its accuracy on the training tumours rises with every level, from 0.923 to 0.998, while its accuracy on the test tumours peaks at 0.944 for depths 3 and 4 and then falls to 0.937. A model that fits its training data better than new data is said to overfit.

Check your understanding (5). What does a reader of a depth-three decision tree see for one prediction?

A. A list of weights

B. The path of at most three questions that leads to the prediction

C. A heat map

D. Nothing

Check your understanding (6). A node of a decision tree holds 40 training tumours, 10 malignant and 30 benign. What is its Gini impurity?

A. 0.375

B. 0.25

C. 0.75

D. 0.5

Python code in the Colab notebook, Section 4.

Open Section 4 of the Colab notebook and run it. `DecisionTreeClassifier(max_depth=3)` grows the tree, and the function `path_of` prints the questions on the path of one patient. The tree of this section asks `area error <= 91.555` where the figure above asks `radius error <= 1.048`. Both questions split off the same single tumour, so the two trees predict alike. A slider then grows the tree to greater depths and shows its number of nodes and its test accuracy.

Accuracy and readability

It is often said that readable models must be less accurate than complex ones. Rudin argues that for many problems with structured data this trade-off is small or absent, and that high-stakes decisions should use models that are readable by design instead of explaining complex models after the fact [16]. A trade-off is a gain in one quality that is paid for with a loss in another, and structured data are data in the form of a table, like the tumour data. The diagnosis case tests the claim: A logistic regression with 31 values, 30 weights and the bias, reaches a test accuracy of 0.986, and a random forest with more than ten thousand decision nodes reaches 0.958. For images, sound and text, deep networks are usually far more accurate, and post-hoc explanations are then the practical option.

A random forest is an ensemble, a model that combines many models, here B decision trees. Each tree is grown on its own random sample of the training cases, drawn with replacement. At each question it may choose only among a few randomly picked features, 5 of the 30 here. The trees therefore differ, and the forest averages their answers:

$$P_{\text{forest}}(y = 1 \mid x) = \frac{1}{B} \sum_{k=1}^{B} P_k(y = 1 \mid x)$$

Here $P_k(y = 1 \mid x)$ is the share of benign tumours in the leaf that the case x reaches in tree k, and the forest predicts benign when the average is above 0.5. The forest of the day has $B = 300$ trees without a depth limit, which together have 10344 nodes. Every single tree is readable. Their average is not: A reader would have to follow 300 paths for one prediction.

To compare models of different kinds, the comparison counts the values a reader must hold in mind to reproduce a prediction by hand: 1 for the threshold rule, 31 for the logistic regression and the number of nodes for a tree or a forest. The table gives the results:

Model	Values to hold in mind	Test accuracy
Threshold rule on worst concave points	1	0.895
Tree of depth 1	3	0.923
Tree of depth 2	7	0.909
Tree of depth 3	13	0.944
Tree of depth 5	29	0.937
Logistic regression	31	0.986
Random forest of 300 trees	10344	0.958

The logistic regression is the most accurate model of the table and is wrong for 2 of the 143 test tumours. The forest holds over three hundred times as many values and is wrong for 6. One tumour is worth 1 / 143 = 0.007, 0.7 percentage points, here, so small differences should not be over-read. On a chart of the table with a logarithmic horizontal axis, every step to the right multiplies the number of values by ten. The upper edge of the points, the best accuracy available for a given number of values, is called the frontier. Where it becomes flat, more complexity buys no more accuracy.

> **Interactive lab, Part E.**
>
> Accuracy against readability on simulated tumours with two measurements: threshold rules, a linear score and trees, each run added as one point to a scoreboard of test accuracy against the number of values a reader must hold in mind. Part E opens at https://utkukose.github.io/xai-veltech-NB-lecture/days/day-01/lab.html#pe.

The figure below repeats the comparison with nine models and a second measure of quality. Accuracy looks only at the side of 0.5 on which a predicted probability falls. The area under the curve, abbreviated AUC, looks at the order of the probabilities. Take every pair of one benign and one malignant test tumour. A pair is ordered correctly when the benign tumour receives the higher probability of benign, and tied when both probabilities are equal:

$$\text{AUC} = \frac{\text{correctly ordered pairs} + \frac{1}{2}\,\text{tied pairs}}{n_1 \times n_0}$$

Here n_1 is the number of benign and n_0 the number of malignant test tumours, so $n_1 \times n_0$ is the number of pairs. An AUC of 1 means that every benign tumour is ranked above every malignant one, and ranking at random gives about 0.5. The full name is area under the receiver operating characteristic curve, a curve under which this share appears as an area.

> **Worked example: an AUC by counting pairs**
>
> A model gives three benign tumours the probabilities of benign 0.9, 0.7 and 0.4, and two malignant tumours 0.6 and 0.1. There are 3 × 2 = 6 pairs. The benign tumours with 0.9 and 0.7 lie above both malignant ones, which gives 2 + 2 = 4 correctly ordered pairs. The benign tumour with 0.4 lies above 0.1 and below 0.6, which gives one more. No pair is tied, and the AUC is (4 + 1) / 6 = 5 / 6 = 0.83.

Interactive lab, Part F.

The AUC formula solved step by step with numbers for five tumours: every pair checked, one benign tumour at a time, with ties counted as one half and sliders for their probabilities. Part F opens at https://utkukose.github.io/xai-veltech-NB-lecture/days/day-01/lab.html#pf.

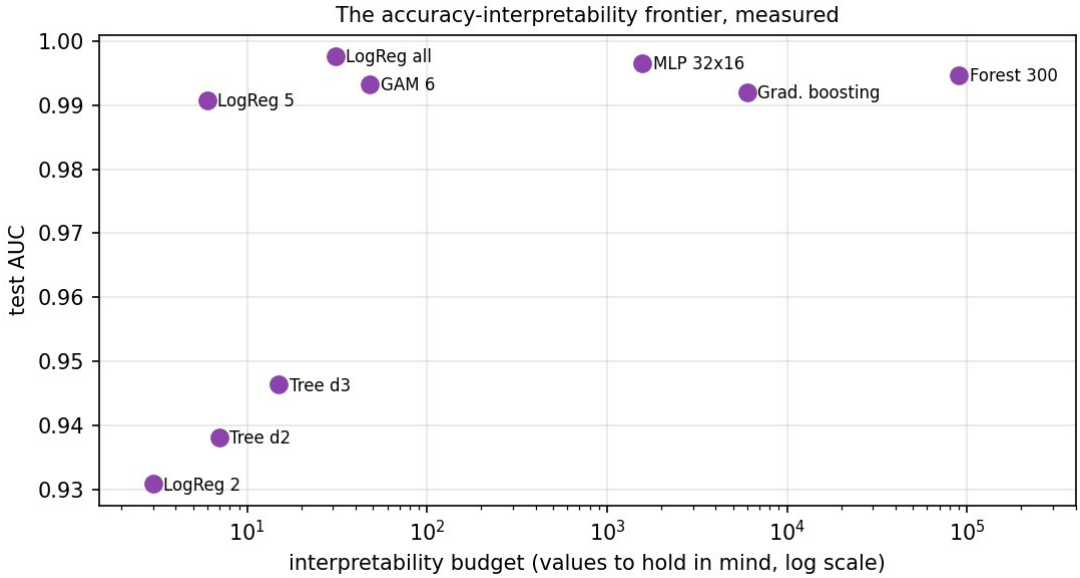


Figure. Test AUC against an estimate of the number of values a reader must hold in mind, for nine models on the tumour data. Beyond a few dozen values, the AUC no longer improves.

The horizontal axis of the figure is an estimate of the number of values and, for the large models, not an exact count: The forest is placed at 90000, while its trees hold about 10000 nodes. LogReg 2, LogReg 5 and LogReg all are logistic regressions on the 2 and the 5 features with the largest weights and on all 30. Tree d2 and Tree d3 are trees of depth 2 and 3. GAM 6 is a generalised additive model, abbreviated GAM, on six features, which the last section of this lecture, Going further, explains. MLP 32x16 is a multilayer perceptron, abbreviated MLP, a neural network with two inner layers of 32 and 16 units. Gradient boosting is an ensemble of 100 small trees in which each tree corrects the errors of the trees before it.

The 143 test tumours form 90 × 53 = 4770 pairs. The logistic regression on all 30 features orders 4759 of them correctly, the highest AUC of the figure, 4759 / 4770 = 0.998. The network reaches 0.997, the forest of 300 trees 0.995, and the two trees and the logistic regression on two features stay below 0.95. Beyond a few dozen values the frontier is flat.

Check your understanding (7). On the tumour data, a logistic regression matches a random forest. What follows?

A. Forests are useless

B. For this problem, a readable model costs no accuracy and is the better choice for a high-stakes decision

C. Logistic regression is always best

D. The data are wrong

Python code in the Colab notebook, Section 5.

Open Section 5 of the Colab notebook and run it. The cell trains the rule, the logistic regression, four trees and the forest, counts for each of them the values a reader must hold in mind and prints the table above. It draws the same numbers as a chart with a logarithmic axis.

What makes a good explanation

An explanation is itself a model of the model, and it can be good or bad. Three properties are checked most often [17, 18]. Fidelity asks whether the explanation agrees with what the model actually does. Stability asks whether similar inputs receive similar explanations. Understandability asks whether the intended reader can use it, which depends on the reader: A data scientist and a patient need different explanations of the same prediction. The animation shows how an explanation can jump while the prediction hardly moves.

For the models of this day, fidelity is perfect by construction: The contributions of a logistic regression add up to its score exactly, and the path of a tree is the computation of the tree. An explanation computed after training reproduces only a part of what the model does, and the last section of this lecture, Going further, measures that part.

Stability can be put into numbers with the smallest explanation of all, the local slope. For a model f with one input x, the slope or derivative $e(x) = df / dx$ states how much the output changes when the input grows a little. Let x' be a slightly changed input. Such a small change is called a perturbation. The relative changes of the prediction and of the explanation are

$$\Delta_f = \frac{|f(x') - f(x)|}{|f(x)|}$$

$$\Delta_e = \frac{|e(x') - e(x)|}{|e(x)|}$$

The vertical bars take the absolute value, the size of a number without its sign, and the Greek letter delta, Δ, stands for a change. An explanation is stable at x when small perturbations give a small Δ_e.

Worked example: the two relative changes

A model gives the output 0.62 at a point and 0.65 after a small perturbation. Its slope is 2.0 at the point and 0.4 after the perturbation. The prediction changes by |0.65 − 0.62| / 0.62 = 0.03 / 0.62 = 0.048, about 5 percent. The explanation changes by |0.4 − 2.0| / 2.0 = 1.6 / 2.0 = 0.8, or 80 percent: It is unstable, although the model itself hardly changed.

In everyday life: a route that jumps

A navigation app may show almost the same arrival time after the car has moved a few metres and yet suggest a completely different route. A driver who sees the route flip back and forth soon stops trusting the app, although its prediction, the arrival time, hardly changed. An explanation that jumps under a tiny change of the input loses trust in the same way.

Animation on the lecture page: A model whose output changes gradually, and the local slope (dashed line) that explains one point. Move the point with the slider and compare the two readouts: the relative change of the prediction and of the explanation, both measured from the starting point.

The model of the animation has a kink at $x = 0.5$, a point where its slope changes abruptly. At the starting point $x = 0.48$ the output is 0.284 and the slope 1.46. A perturbation of +0.020 changes the output by 10.6 percent and the slope by 6 percent. One step of the slider further, at +0.022, the output has changed by 10.7 percent and the slope by 91 percent. Models built from trees have many such abrupt changes, because their output moves in steps.

Understandability has no formula, but one ingredient can be counted: An explanation is sparse when it uses few inputs. The explanation of test patient 5 above names only the six largest of the 30 contributions for this reason.

> Check your understanding (8). Two almost identical patients receive very different explanations. Which property is violated?
>
> A. Fidelity
>
> B. Stability
>
> C. Accuracy
>
> D. Sparsity

Accurate for the wrong reason

Geirhos and colleagues describe shortcut learning: A model finds a simple cue in the training data that predicts the label without being the reason for it [3]. A well-known kind of case in medical imaging is a ruler, a marker or a scanner label that appears more often in images of one class. The lesion images of the diagnosis case are such a case: synthetic tumour images in which a bright marker in the corner appears in almost every image with a lesion.

Synthetic means that a program draws the images, so the true reason for every label is known. Each image has 32 by 32 pixels, 1024 grey values between 0 for black and 1 for white. The background is random noise around 0.30 with six soft bright patches at random places. A patch is a bump of brightness that fades within a few pixels of its centre. With a probability of one half the image receives a lesion, a damaged region of tissue, drawn as one more patch near the centre, and the label $y = 1$. The lesion has a brightness of 0.25 at its centre and the background patches between 0.10 and 0.22, so a lesion is hard to tell from the background. The marker is a white square of 4 by 4 pixels near the top left corner. It is added to 97 percent of the images with a lesion and to 3 percent of the images without one.

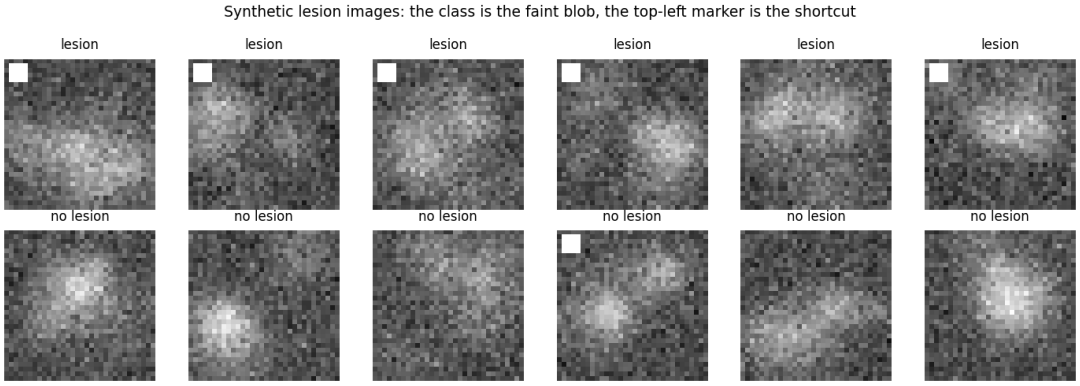


Figure. Synthetic lesion images. The class is defined by a faint blob; in the training data, a bright corner marker almost always accompanies it.

The top row of the figure holds six images with a lesion, and five of them carry the marker. The bottom row holds six images without a lesion, and one of them carries the marker. The lesion is hard to see, and the marker cannot be missed.

The model is a logistic regression on the raw pixels, with one weight per pixel, trained on 2400 images. A simple model reaches 96 percent accuracy on test images of the same kind and only 50 percent, the level of guessing, when the marker no longer carries information. Nothing in the accuracy warned of this. The first number comes from 600 images that were split off at random from the same data, so the marker works in them as in training. The second comes from 800 new images in which the marker appears in half of the images of either class. A random split cannot reveal a shortcut. It shows only on data in which the cue and the label have been separated.

An explanation that shows where the model looks reveals it at once. Because the model is linear, the size of a weight, its absolute value $|w_k|$, shows how strongly the model listens to pixel k. These sizes can be drawn as an image, and the share that falls on the 16 pixels of the marker is

$$\text{marker share} = \frac{\sum_{k \in \text{marker}} |w_k|}{\sum_{k=1}^{1024} |w_k|}$$

The sign $\in$ reads *in*: The upper sum runs over the pixels of the marker only. The marker covers 16 / 1024 = 0.016 of the image, 1.6 percent, and the shortcut model puts 4.7 percent of its weight there, 4.7 / 1.6 = 2.9, about three times as much. Its predictions agree with the presence of the marker in every one of the 600 test images: In effect it has learned the rule that a marker means a lesion.

> **In everyday life: Clever Hans**
>
> Around 1900 a horse in Berlin, known as Clever Hans, seemed to solve sums: It tapped its hoof until it reached the right answer. A psychologist found that the horse watched the questioner, who relaxed without noticing when the right number of taps was reached. With a questioner who did not know the answer, the horse failed. It was right for the wrong reason, like the lesion model with its marker, and only a test that separated the cue from the answer revealed it.

Ribeiro and colleagues built such a case on purpose with photographs: a classifier that told huskies from wolves, trained on pictures in which every wolf stood on snow and no husky did [19]. Its explanations showed that it looked at the snow. The animation repeats the idea with drawn photos and a model whose score is $s = 4\,\text{snow} + 0.5\,\text{wolf} - 2.2$, where each of the two features is 1 or 0 and the model answers wolf when $s > 0$. With photos like the training data it is right for all eight. With swapped backgrounds a husky on snow scores $4 - 2.2 = 1.8$ and a wolf on grass $0.5 - 2.2 = -1.7$, and the model is wrong for all eight, because $4 / (4 + 0.5) = 0.89$ of its weight lies on the background.

Animation on the lecture page: Husky or wolf? A classifier that is right for the wrong reason. Switch between photos like the training data and photos with swapped backgrounds, and show where the model looks.

The repair is in the data: When the model is trained on images in which the marker appears equally often in both classes, it has to look for the lesion itself. The repaired model has a marker share of 1.4 percent and reaches 61 percent on both test sets, a lower accuracy that no longer depends on the cue. A linear model on raw pixels finds that much harder, which is why Day 3 turns to convolutional networks. These are neural networks for images that search for a pattern at every position, while one fixed weight per pixel can only register that the centre of an image is a little brighter.

Check your understanding (9). A model scores 96 percent on test data and 50 percent once a marker stops carrying information. What did it learn?

A. The lesion

B. A shortcut: the marker

C. Nothing

D. The noise

Python code in the Colab notebook, Section 6.

Open Section 6 of the Colab notebook and run it. The function `make_lesion_images` draws the synthetic images, and `marker_share` is the formula above, computed from the sizes of the weights, `np.abs(m.coef_[0])`. The cell trains the shortcut model and the repaired model, prints their accuracies and marker shares and draws the two maps of weights side by side.

The diagnosis case, stage 1: readable models and a shortcut found

The methods of this day now come together in the first stage of the project. The tumour data receive three readable models, a threshold rule, a logistic regression and a tree, and a random forest for comparison, and the lesion images a linear model whose weights can be drawn as a map. The figure shows the audit of this stage, and the table collects what it has shown.

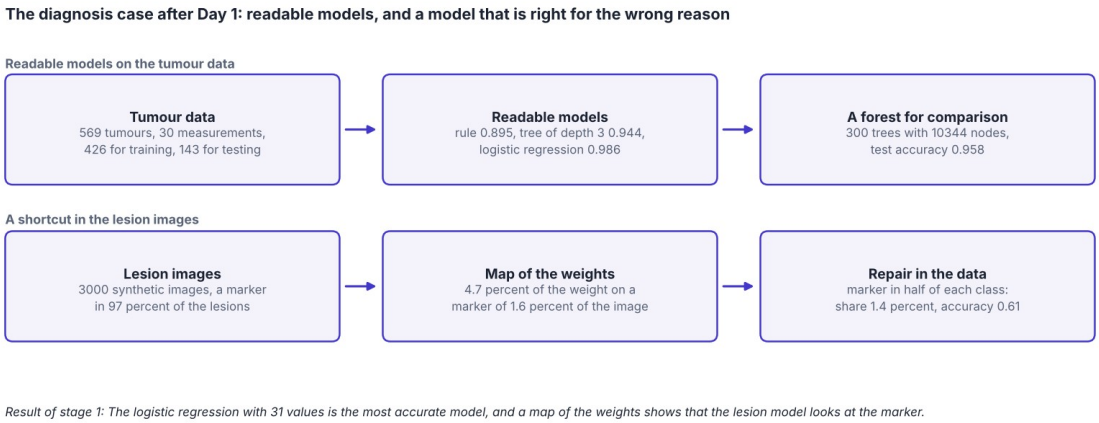


Figure. The audit after Day 1: readable models of the tumour data, and the shortcut of the lesion images found in a map of the weights.

Question	Answer of stage 1	Section
Can a rule of one sentence diagnose?	Benign when the worst concave points are at most 0.14: 89.5 percent of the test tumours right, 8 of 53 malignant tumours missed	Models that explain themselves
Which measurements drive the logistic regression?	`worst texture` with −1.25, then `radius error` and five weights between −0.96 and −0.92, at a test accuracy of 0.986	Models that explain themselves

Question	Answer of stage 1	Section
Can one prediction be read?	Test patient 10 follows three questions of the tree to a leaf in which 9 of 12 training tumours are benign	Models that explain themselves
Does readability cost accuracy here?	No: 31 values reach 0.986, and the forest with 10344 nodes 0.958	Accuracy and readability
Are the explanations stable?	The contributions and the path are exact, while a local slope can change by 91 percent when the prediction changes by 11 percent	What makes a good explanation
Is the lesion model right for the right reason?	No: 4.7 percent of its weight lies on a marker of 1.6 percent of the image, and its accuracy falls from 96 to 50 percent once the marker carries no information	Accurate for the wrong reason

Two questions remain open for the next days. The random forest cannot be read, and Day 2 explains it with methods that only ask it for predictions. The linear model of the lesion images is weak, and Day 3 trains a convolutional network on the same images and asks where it looks.

Explanations in your field

The same questions apply in every field that uses data to decide: Which measurements matter, can the model be read, and does it rely on something it should not? The application of the day loads a problem from medicine, chemistry, manufacturing, energy or civil engineering and builds a logistic regression, a tree of depth 3 and a random forest for it. Every problem has two classes, and the class with the label 1 is called the positive class.

Field	Cases	Features	Positive class
medicine	569 tumours, real data	the 30 measurements of the cell nuclei	benign tumour
chemistry	178 wines, real data	13 chemical measurements, such as alcohol and proline	wine of the first of three grape varieties, called cultivars
manufacturing	1500 simulated weld seams	current in ampere, voltage in volt, travel speed in millimetres per second, gap in millimetres, plate temperature in degrees Celsius	sound weld
energy	1500 simulated samples of transformer oil	five dissolved gases in parts per million: hydrogen (`H2`), methane (`CH4`), acetylene (`C2H2`), ethylene (`C2H4`) and carbon monoxide (`CO`)	fault in the transformer
civil	1500 simulated concrete mixes	cement, water, slag and superplasticizer in kilograms per cubic metre, age in days	mix that reaches a strength of 30 megapascals

In the three simulated fields a known rule creates the label: A score s is computed from the features, random noise ε with the mean 0 is added, and the result is compared with a limit. The letter ε is the Greek epsilon. For the weld seams the score is

$$s = 0.04\,(\text{current} - 180) - 0.9\,(\text{speed} - 6)$$
$$- 1.8\,(\text{gap} - 0.8) + 0.02\,(\text{temperature} - 40) + \varepsilon$$

A weld is sound when s is positive. The noise has the standard deviation 1, and the voltage does not enter the rule. For the transformer oil, with square brackets for the concentration of a gas, the score is

$$s = 0.8 \ln[C_2H_2] + 0.6 \ln \frac{[H_2]}{30} + 0.5 \ln \frac{[C_2H_4]}{12} - 0.2 \ln \frac{[CO]}{150} + \varepsilon$$

A fault is present when this score is above 0.6. The noise has the standard deviation 0.6, and methane does not enter the rule. For the concrete mixes the rule gives a strength in megapascals:

$$\text{strength} = 0.09\,\text{cement} - 0.15\,\text{water} + 0.04\,\text{slag}$$
$$+ 0.7\,\text{superplasticizer} + 9 \ln \frac{\text{age}}{7} + \varepsilon$$

A mix is positive when its strength is above 30, and the noise has the standard deviation 4. Because of the noise, no model can be perfect on these data. The rule itself, applied without its noise, is right for only 79 percent of the weld seams, 77 percent of the oil samples and 92 percent of the concrete mixes in the test data. A model that comes close to these values has found the rule, and its weights show it. For the weld seams the logistic regression gives +1.33 to the current, −1.80 to the speed, −1.65 to the gap, +0.39 to the plate temperature and −0.02 to the voltage.

A field that every reader knows is the e-mail inbox. A spam filter is a classifier with the two classes spam and not spam. Its weights are a global explanation that a reader can check word by word, and the contributions of the words to one e-mail are a local explanation of one decision.

> **Interactive lab, Part G.**
>
> Eight e-mails and a spam filter with five words: Set the weights so that every e-mail lands in the right folder with as few words as possible, and watch the e-mails move between the inbox and the spam folder. Part G opens at https://utkukose.github.io/xai-veltech-NB-lecture/days/day-01/lab.html#pg.

> **Python code in the Colab notebook, Section 7.**
>
> Open Section 7 of the Colab notebook. Its first line is a switch, `FIELD = "medicine"`. Replace the word by `chemistry`, `manufacturing`, `energy` or `civil` and run the section. The function `load_field` loads or simulates the data of that field, and the cell trains a logistic regression, a tree of depth three and a random forest on them and shows the weights, the tree and the test accuracies.

Going further (optional)

Two ideas extend the day for readers who want more depth. A generalised additive model replaces each weight of a logistic regression by a smooth curve, one per input, and adds the curves up, so that it captures nonlinear effects and stays readable [20]:

$$g\left(\mathbb{E}[y \mid x]\right) = \beta_0 + \sum_{j=1}^{d} f_j(x_j)$$

Here $E[y \mid x]$ is the expected value of the label given the inputs, which for a label of 0 or 1 is the probability p of the class 1. The function g is called the link function and is here the log-odds, $g(p) = \ln(p / (1 - p))$. The Greek letter beta with the index 0, β_0, is the bias, and f_j is the curve of feature j, called its shape function. A logistic regression is the special case in which every shape function is a straight line, $f_j(x_j) = w_j \tilde{x_j}$. An effect is nonlinear when its curve is not a straight line.

The model of this section builds each shape function as a weighted sum of seven splines, smooth bump-shaped functions that each cover one part of the range of a feature. The weights are found like those of a logistic regression, with $C = 0.5$. With the four features `worst radius`, `worst texture`, `worst concave points` and `mean smoothness`, the model has $4 \times 7 = 28$ weights and a bias, and it classifies 138 of the 143 test tumours correctly, an accuracy of $138 / 143 = 0.965$. For each feature, a curve shows the log-odds of benign while the other three are held at their median, the middle value of the training tumours. The curve of `worst radius` changes little up to about 12, falls steeply and crosses zero near 18.6, where the prediction changes from benign to malignant. A single weight could not show such a threshold.

The quality of an explanation can also be measured. For a local linear explanation g of a model f around a case x, fidelity is the share of the model's behaviour near x that the explanation reproduces, and stability compares the change of the explanation with the change of the input [17]. Here f is the random forest, and g is no longer the link function but the explanation, a small model of its own. Day 2 develops it into the method of local interpretable model-agnostic explanations, abbreviated LIME [19]. The explanation draws 400 sample points z around the patient x by adding random noise with a quarter of the standard deviation s_j to every feature, and asks the forest for its probability of benign $f(z)$ at each point. A sample point close to the patient receives a large weight:

$$u_j = \frac{z_j - x_j}{s_j}, \qquad \pi_x(z) = \exp\left(-\frac{\|u\|^2}{2h^2}\right)$$

Here u_j is the offset of the sample point in feature j, counted in standard deviations, and $\exp(v)$ is another way to write e^v. The double bars are the length of the vector u, the square root of the sum of its squared entries, and h is the median of the 400 lengths. The Greek letter pi in $\pi_x(z)$ names the weight. The explanation is the linear model $g(z) = a_0 + a_1 u_1 + \ldots + a_d u_d$ that fits the answers of the forest best. Its coefficients make the weighted sum of the squared differences between $f(z)$ and $g(z)$, plus the sum of the squares of a_1 to a_d, as small as possible. Such a fit with a penalty is called ridge regression. The coefficients a_1 to a_d are the explanation $e(x)$: Each a_j estimates how much the forest's probability changes near this patient when feature j grows by one standard deviation. The fidelity is

$$\mathrm{Fid}(g, f, x) = 1 - \frac{\sum_z \pi_x(z)\,(f(z) - g(z))^2}{\sum_z \pi_x(z)\,(f(z) - \bar{f})^2}$$

The numerator adds up the weighted squared errors of the explanation over the sample points. The denominator does the same for the plainest explanation possible, the constant $\bar{f}$, the weighted mean of the answers of the forest. A fidelity of 1 means that the explanation reproduces the forest exactly at the sample points, and 0 that it is no better than the constant. For test patient 3, the fidelity is 0.638: The explanation reproduces 64 percent of the variation of the forest around this patient.

Worked example: a fidelity by hand

Three sample points have the weights 1, 1 and 0.5. The forest answers 0.8, 0.6 and 0.2, and the linear explanation 0.7, 0.6 and 0.4. The weighted mean of the forest is (1 × 0.8 + 1 × 0.6 + 0.5 × 0.2) / 2.5 = 1.5 / 2.5 = 0.6. The errors of the explanation are 0.8 − 0.7 = 0.1, 0.6 − 0.6 = 0 and 0.2 − 0.4 = −0.2, and the distances of the forest from its mean are 0.8 − 0.6 = 0.2, 0 and 0.2 − 0.6 = −0.4.

The numerator is $1 \times 0.1^2 + 1 \times 0^2 + 0.5 \times 0.2^2 = 0.01 + 0 + 0.02 = 0.03$, and the denominator is $1 \times 0.2^2 + 1 \times 0^2 + 0.5 \times 0.4^2 = 0.04 + 0 + 0.08 = 0.12$. The fidelity is 1 − 0.03 / 0.12 = 1 − 0.25 = 0.75.

For stability, the general measure asks for the worst case among all inputs x' within a distance ε of x:

$$L(x) = \max_{\|x'-x\| \le \varepsilon} \frac{\|e(x') - e(x)\|}{\|x' - x\|}$$

It is the largest change of the explanation per unit of change of the input. A plainer average serves here. The patient is perturbed $K = 8$ times with noise of 2 percent of the standard deviation of every feature, the explanation is computed again each time, and the mean relative change is reported:

$$\bar{\Delta}_e = \frac{1}{K} \sum_{k=1}^{K} \frac{\|e(x'_k) - e(x)\|}{\|e(x)\|}$$

This is the measure Δ_e of the animation for an explanation with 30 numbers, and the result is 29.2 percent. Each repetition changes two things at once, the patient and the 400 random sample points. With the same patient and new sample points only, the mean change is 29.0 percent, and with the perturbed patient and unchanged sample points it is 6.7 percent. Most of the instability of this explanation comes from its own random sampling, a reminder that post-hoc explanations need checks of their own. Days 2 and 4 return to both measures.

Python code in the Colab notebook, Section 8.

Open Section 8 of the Colab notebook, which is optional, and run it. The first cell fits the generalised additive model with `SplineTransformer` and draws its curves. The function `local_explanation` then samples 400 points around one patient, fits the weighted linear explanation with `Ridge` and computes its fidelity, and the loop over `seed=k` measures its stability.

Answers to the knowledge checks

Quick check. `False`. The first comparison is true and the second is false, and `and` is true only when both sides are true.

1. B. Deep learning is a part of machine learning: The network learns its weights from examples, and the methods of the course apply to it.

2. B. Accuracy measures how often a model is right, not why.

3. B. It explains one decision and is computed after training.

4. C. The contributions are −2 × 1.0 = −2 and 1 × 0.5 = 0.5, the score is 0.5 − 2 + 0.5 = −1, and $1 / (1 + e^1) = 1 / (1 + 2.718) = 1 / 3.718 = 0.27$.

5. B. Each prediction is the conjunction of the tests on its path: All of them hold at once.

6. A. The share of benign is 30 / 40 = 0.75, and $G = 2 \times 0.75 \times 0.25 = 0.375$.

7. B. When a readable model is as accurate, its transparency comes for free.

8. B. Stability requires similar inputs to receive similar explanations.

9. B. Its accuracy depended on a cue that is not the reason for the label.

References

[1] Arrieta, A. B., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., Chatila, R., & Herrera, F. (2020). Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. Information Fusion, 58, 82-115.

[2] Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. arXiv preprint arXiv:1702.08608.

[3] Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., & Wichmann, F. A. (2020). Shortcut learning in deep neural networks. Nature Machine Intelligence, 2(11), 665-673. https://doi.org/10.1038/s42256-020-00257-z

[4] Wolberg, W., Mangasarian, O., Street, N., & Street, W. (1993). Breast Cancer Wisconsin (Diagnostic) [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5DW2B

[5] Van Rossum, G., & Drake, F. L. (2009). Python 3 Reference Manual. CreateSpace.

[6] European Parliament and Council of the European Union. (2024). Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, L series, 12 July 2024. http://data.europa.eu/eli/reg/2024/1689/oj

[7] Kose, U., Şengöz, N., Chen, X., & Marmolejo Saucedo, J. A. (Eds.). (2024). Explainable Artificial Intelligence (XAI) in Healthcare. CRC Press.

[8] Mangasarian, O. L., Street, W. N., & Wolberg, W. H. (1995). Breast cancer diagnosis and prognosis via linear programming. Operations Research, 43(4), 570-577. https://doi.org/10.1287/opre.43.4.570

[9] Lipton, Z. C. (2018). The mythos of model interpretability. Communications of the ACM, 61(10), 36-43.

[10] Ortigossa, E. S., Gonçalves, T., & Nonato, L. G. (2024). Explainable Artificial Intelligence (XAI) - from theory to methods and applications. IEEE Access, 12, 80799-80846.

[11] Simonyan, K., Vedaldi, A., & Zisserman, A. (2014). Deep inside convolutional networks: Visualising image classification models and saliency maps. In Workshop at the 2nd International Conference on Learning Representations (ICLR 2014). arXiv:1312.6034. https://arxiv.org/abs/1312.6034

[12] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In Proceedings of the IEEE International Conference on Computer Vision (ICCV 2017) (pp. 618-626). https://doi.org/10.1109/ICCV.2017.74

[13] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In Advances in Neural Information Processing Systems 30 (NeurIPS 2017) (pp. 4765-4774). https://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions

[14] Wachter, S., Mittelstadt, B., & Russell, C. (2018). Counterfactual explanations without opening the black box: Automated decisions and the GDPR. Harvard Journal of Law & Technology, 31(2), 841-887. https://jolt.law.harvard.edu/volumes/volume-31

[15] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[16] Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. Nature Machine Intelligence, 1(5), 206-215.

[17] Hedström, A., Weber, L., Krakowczyk, D., Bareeva, D., Motzkus, F., Samek, W., Lapuschkin, S., & Höhne, M. M.-C. (2023). Quantus: An explainable AI toolkit for responsible evaluation of neural network explanations and beyond. Journal of Machine Learning Research, 24(34), 1-11. https://jmlr.org/papers/v24/22-0142.html

[18] Bhatt, U., Xiang, A., Sharma, S., Weller, A., Taly, A., Jia, Y., Ghosh, J., Puri, R., Moura, J. M. F., & Eckersley, P. (2020). Explainable machine learning in deployment. In Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '20) (pp. 648-657). ACM.

[19] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?" Explaining the predictions of any classifier. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16) (pp. 1135-1144). ACM.

[20] Lou, Y., Caruana, R., & Gehrke, J. (2012). Intelligible models for classification and regression. In Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '12) (pp. 150-158). ACM. https://doi.org/10.1145/2339530.2339556

[21] Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., Willing, C., & Jupyter development team. (2016). Jupyter Notebooks: A publishing format for reproducible computational workflows. In F. Loizides & B. Schmidt (Eds.), Positioning and Power in Academic Publishing: Players, Agents and Agendas (pp. 87-90). IOS Press. https://doi.org/10.3233/978-1-61499-649-1-87

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye

Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University

Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 · utkukose.com · github.com/utkukose

utkukose@sdu.edu.tr · utku.kose@und.edu · ukose@up.edu.mx · utkukose@gmail.com

This course is updated in line with current developments in the field. First version: October 2026. Last update: October 2026.